

Review 3:

1.

```
/**
```

```
 * Returns the product of the digits of {@code n}.
```

```
 *
```

```
 * @param n
```

```
 *           {@code NaturalNumber} whose digits to multiply
```

```
 * @return the product of the digits of {@code n}
```

```
 * @clears n
```

```
 * @ensures productOfDigits1 = [product of the digits of n]
```

```
 */
```

```
private static NaturalNumber productOfDigits1(NaturalNumber n) {
```

```
    NaturalNumber product = new naturalNumber2(1);
```

```
    While(!n.isZero()){
```

```
        NaturalNumber digit = new NaturalNumber2(n.divideBy10());
```

```
        Product.multiply(digit);
```

```
    }
```

```
    Return product;
```

```
}
```

```
/**
```

```
 * Returns the product of the digits of {@code n}.
```

```

*
* @param n
*           {@code NaturalNumber} whose digits to multiply
* @return the product of the digits of {@code n}
* @ensures productOfDigits2 = [product of the digits of n]
*/

private static NaturalNumber productOfDigits2(NaturalNumber n) {
    NaturalNumber copt = new naturalNumber2(n);
    NaturalNumber product = new naturalNumber2(1);
    While(!copy.isZero()){
        NaturalNumber digit = new NaturalNumber2(n.divideBy10());
        Product.multiply(digit);
    }
    Return product;
}

2.

/**
 * Reports the value of {@code n} as an {@code int}, when {@code
n} is
 * small enough.
 *

```

```

* @param n
*
*         the given {@code NaturalNumber}
*
* @return the value
*
* @requires n <= Integer.MAX_VALUE
*
* @ensures toInt = n
*
*/

private static int toInt(NaturalNumber n) {
    assert n.compareTo(new NaturalNumber2(Integer.MAX_VALUE))
        : "Violation of: n<= Integer.MAX_VALUE":
        Return n.toInt;
}

```

3.

```

/**
 * Reports whether the given tag appears in the given {@code
XMLTree}.
 *
 * @param xml
 *
 *         the {@code XMLTree}
 *
 * @param tag
 *
 *         the tag name
 *
 * @return true if the given tag appears in the given {@code

```

XMLTree},

* false otherwise

* @ensures <pre>

* findTag =

* [true if the given tag appears in the given {@code XMLTree},
false otherwise]

* </pre>

*/

private static boolean findTag(XMLTree xml, String tag) {

boolean found = xml.label().equals(tag));

for(int I = 0; I <xml.numberOfChildren() && !found; i++){

 found = finaTag(xml.child(i),tag);

}

Return found;

}

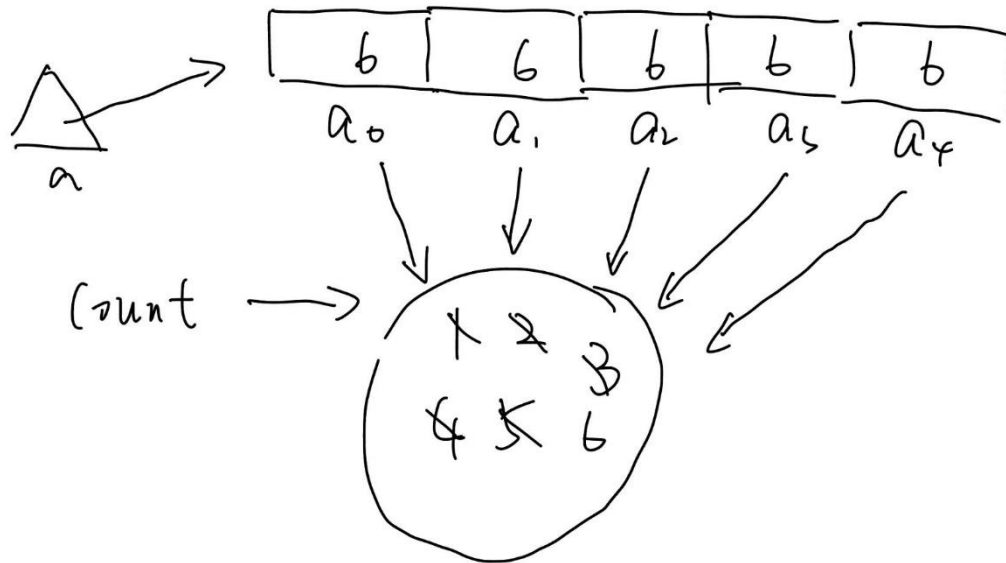
4.

- i. design-by-contract: A programming methodology where software components interact based on explicitly defined contracts that specify preconditions, postconditions, and invariants.
- ii. Precondition: A condition that must be true before a function or method executes to ensure correct behavior.
- iii. Postcondition: A condition that must be true after a function or

method has executed, assuming the precondition was met.

- iv. Testing: The process of executing a program with different inputs to verify that it behaves correctly.
- v. Debugging: The process of identifying, analyzing, and fixing errors or bugs in a program.
- vi. parameter mode: Specifies how a parameter is passed to a method.
- vii. Clears: A parameter mode that ensures the parameter is reset to a default state after the function executes.
- viii. Replaces: A parameter mode where the output value completely replaces the input value.
- ix. Restores: A parameter mode ensuring that a parameter's value is unchanged after method execution.
- x. Updates: A parameter mode where the value is modified based on the previous value and method logic. (has a value that might be changed from its incoming value)
- xi. immutable type: A data type whose instances cannot be modified after creation.
- xii. primitive type: The built-in types.
- xiii. reference type: All other types.
- xiv. Object: An instance of a class that contains state and behavior.
- xv. Aliasing: When multiple variables reference the same object in memory.

- xvi. declared type/static type: The type of a variable as defined
- xvii. object type/dynamic type: The actual type of an object at runtime, which may be a subclass of the declared type.
- xviii. Implements: A keyword on Java used when a class provides the implementation for an interface.
- xix. Extends: The name of the inheritance relationship.
- xx. method overriding: Provide two or more methods with the same name but different formal parameters
- xxi. subinterface/derived interface/child interface: An interface that extends another interface, inheriting its method signatures.
- xxii. superclass/base class/parent class: A class from which other classes inherit properties and behaviors.
- xxiii. Polymorphism: The ability of different classes to be treated as instances of a common superclass.
- xxiv. Recursion: A technique where a function calls itself to solve a smaller instance of the problem.



They will array to the same count, so that we need to create a new count to fix it.

6. `NaturalNumber` extends `NaturalNumber-Kernel` and the `NaturalNumber-Kernel` also extends `standard` so that `NaturalNumber` has all the functions or methods in the `standard`.

7. `C3` has implemented `I2` so that it can use `I2` and `I2` also extend `I1` so that `C3` also can use or implement `I1`.

`C4` has extended `C3` so that `C4` has all the methods in `C3` and `C3` has implemented `I2` so that `C4` can also implement `I2`.